

Implementation of 8-bit Booth's Multiplier using Semi-Custom VLSI Design Flow

Name:-

B.Naga Praveen(123EC0015)

Aim

To design, simulate, synthesize, and implement an 8-bit Booth's Multiplier using Verilog HDL and perform physical implementation using Cadence Genus and Innovus VLSI design tools.

Apparatus Required

- Cadence Genus (Logic Synthesis)
- Cadence Innovus (Place and Route)
- Cadence SimVision (Simulation)
- Verilog HDL
- Linux Workstation

Theory

- Booth's algorithm helps multiply signed binary numbers using fewer addition and subtraction steps.
- It works by examining pairs of bits in the multiplier and deciding whether to add, subtract, or do nothing with the multiplicand, followed by right-shifting bits.
- This reduces the total number of operations compared to simple multiplication, saving hardware and power.
- In semi-custom VLSI design flow, this algorithm is implemented by designing custom digital blocks (like adders, shifters, and control logic) using standard cell libraries.
- The multiplier circuit is then synthesized, placed, and routed on silicon using CAD tools, optimizing for speed, area, and power.

Booth Algorithm Rules

$$(Q_0, Q_{-1}) = \begin{cases} 01 \Rightarrow A = A + M \\ 10 \Rightarrow A = A - M \\ 00 \text{ or } 11 \Rightarrow \text{No Change} \end{cases}$$

Booth multiplication is efficient for signed numbers.

It reduces number of additions/subtractions compared to normal binary multiplication.

It works well even when large consecutive 1's appear in the multiplier.

- A = Accumulator register
- M = Multiplicand
- Q = Multiplier
- Q = Stored previous least significant bit
- After the chosen operation, perform an Arithmetic Right Shift (ARS) on the combined register(A,Q,Q₁)
- Repeat this procedure for each bit of the multiplier.
- After all iterations, the final product is obtained as: Product = A parallel Q

DISADVANTAGES

- It is less efficient if the multiplier has alternating 1s and 0s (like 1010).
- The number of addition and subtraction steps can vary, making design complex.
- It is harder to understand and implement compared to simple multiplication.
- Hardware using it may consume more power and space.
- Works mainly for signed numbers and needs changes for unsigned numbers.

Advantages

- Efficient for signed multiplication
- Reduces number of additions/subtractions
- Reduces hardware complexity

Procedure

1. Write Verilog code for Booth's Multiplier.
2. Write a testbench and verify the output using simulation.
3. Apply timing constraints using .sdc.

4. Synthesize the circuit using Cadence Genus.
5. Import synthesized netlist into Cadence Innovus.
6. Perform floorplanning, placement, CTS, routing, and signoff checks.

Simulation Waveform

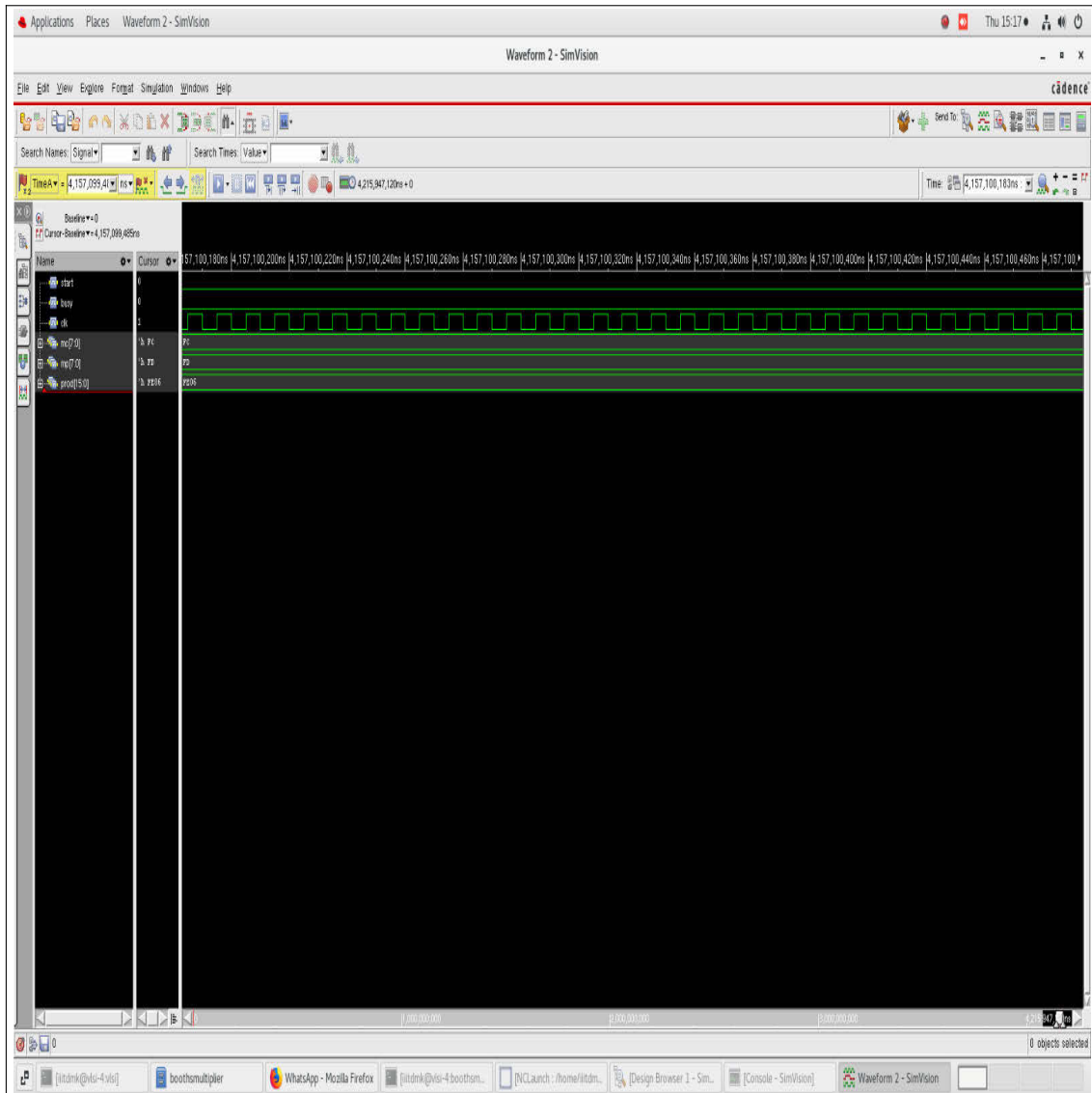


Figure 1: Simulation Waveform of Booth Multiplier

Explanation: The waveform shows that the Booth multiplier is working correctly. When you give inputs (multiplicand and multiplier), the output product becomes stable after the multiplier does a series of adding, subtracting, and shifting steps one by one. This means the Verilog code runs the Booth algorithm properly before the final chip design process (synthesis).

Synthesized Gate-Level Schematic

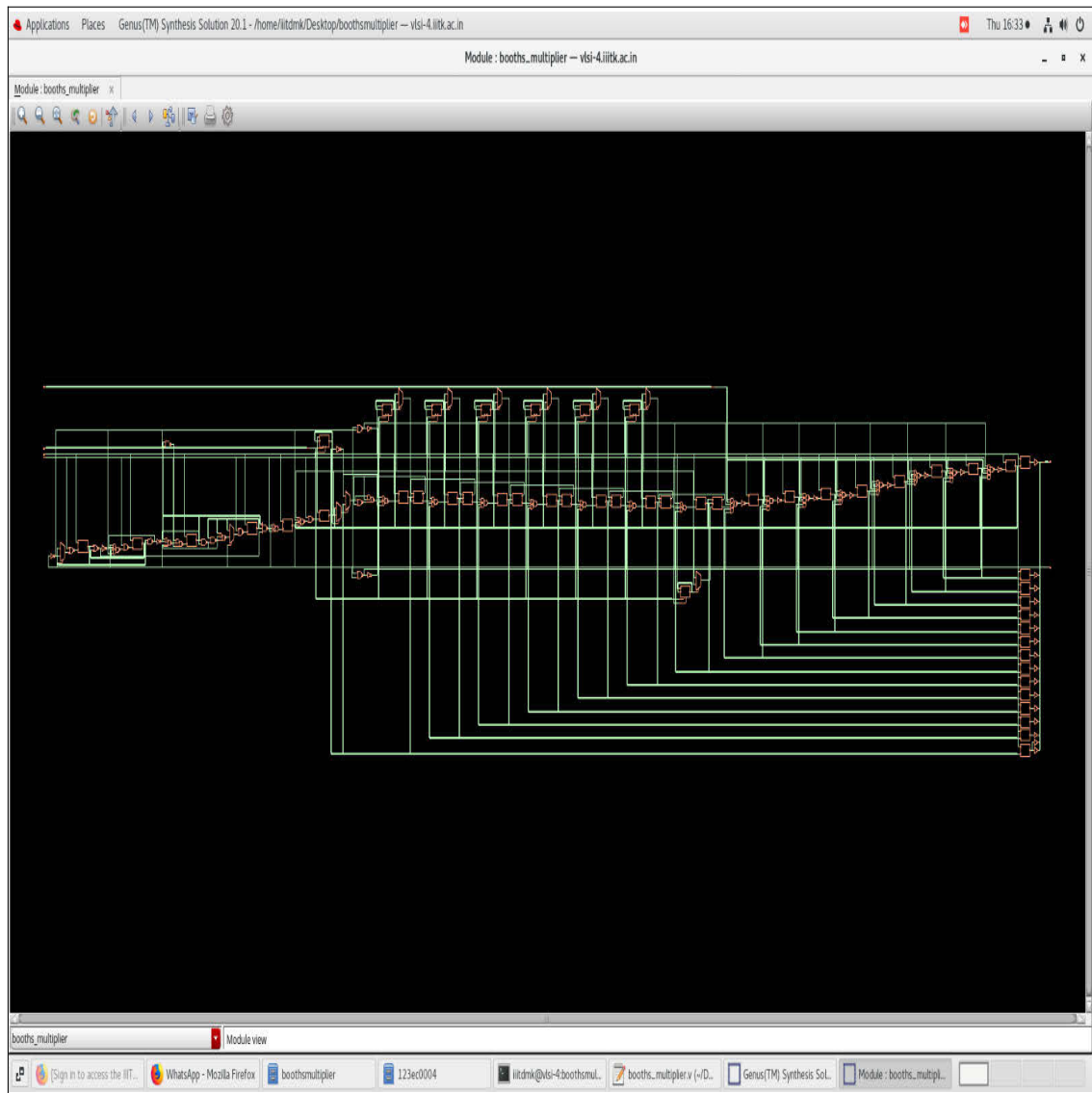


Figure 2: Gate-Level Schematic from Genus

Explanation: This schematic is a simple drawing that shows how the Booth multiplier hardware looks after the Verilog code is changed into actual circuit parts during synthesis. It includes important parts like:

- Registers: These hold numbers temporarily.
- Adders: These add numbers during multiplication.
- Arithmetic shifters: These shift bits left or right, which is part of the multiplication process.
- Control logic: This manages when and how these parts work together.
- Together, these parts create the path that data follows to complete the Booth multiplication step by step inside the chip.

Timing Report

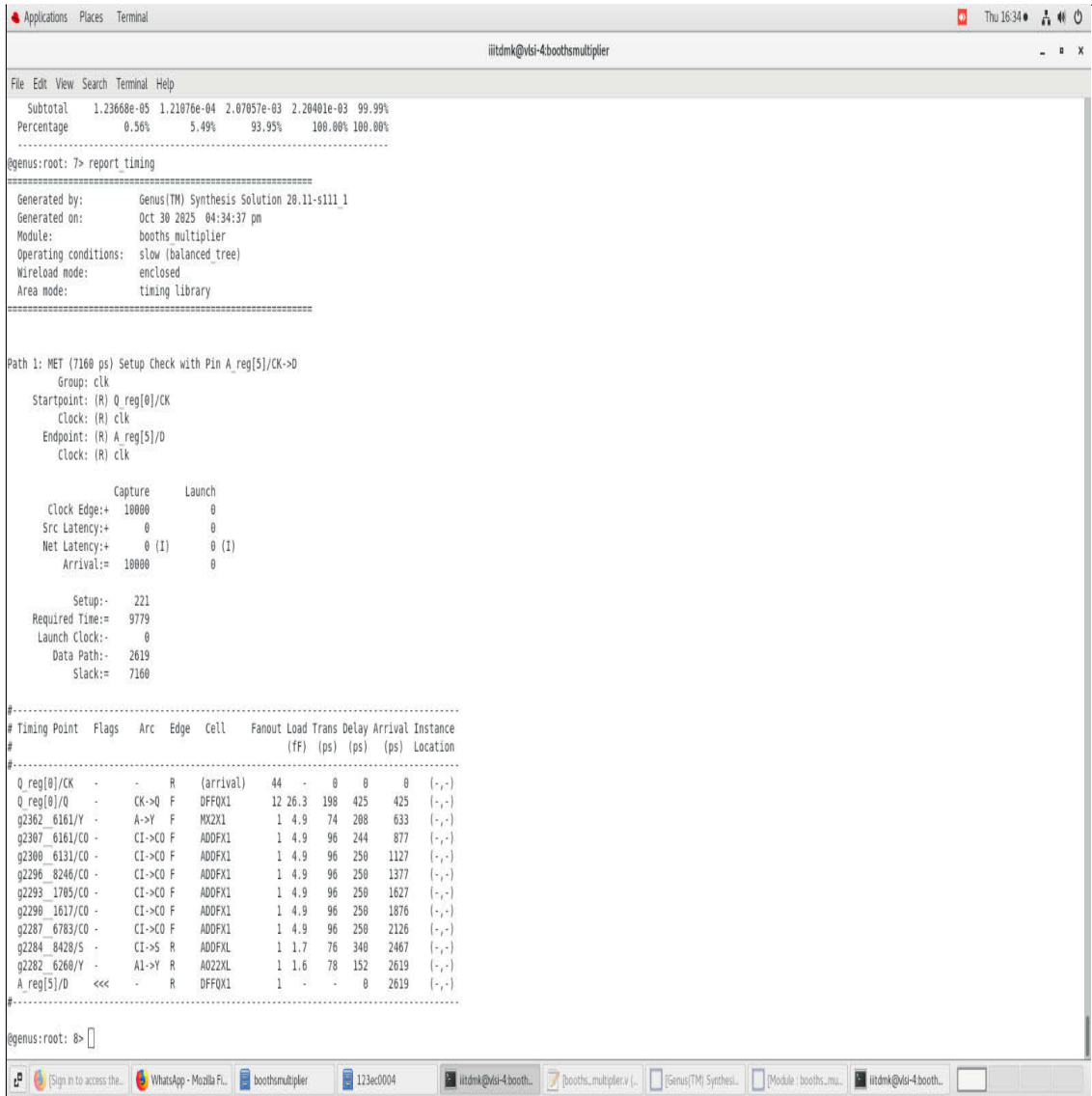


Figure 3: Timing Analysis Report

Explanation:

The timing report shows how well the circuit meets its timing goals. It includes:

- Setup time: The minimum time input signals must be stable before the clock.
- Hold time: The minimum time input signals must stay stable after the clock.
- Slack: The extra time available beyond what's needed.

If the slack is positive, it means the design works on time and can run safely at the chosen clock speed without errors. This means the circuit is reliable and runs fast enough.

Power and Area Report

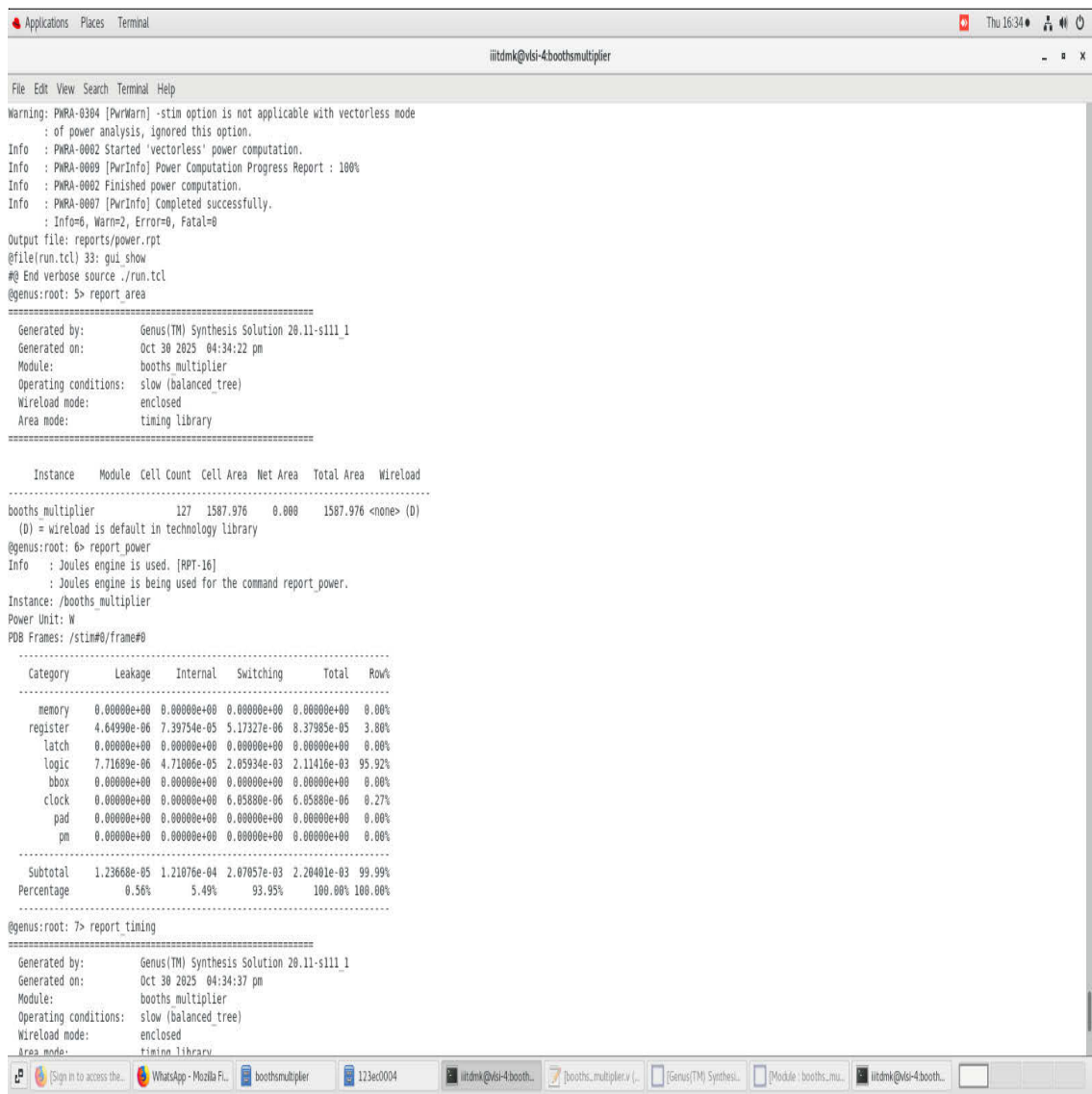


Figure 4: Power and Area Report

Explanation: The report displays total power consumption, leakage power, and silicon area used. Booth's algorithm reduces switching activity, improving power efficiency while maintaining acceptable area utilization.

Placed Layout

Explanation: This placement step arranges the standard cells to minimize routing complexity and reduce wire delay. Proper placement ensures better performance and lower power.

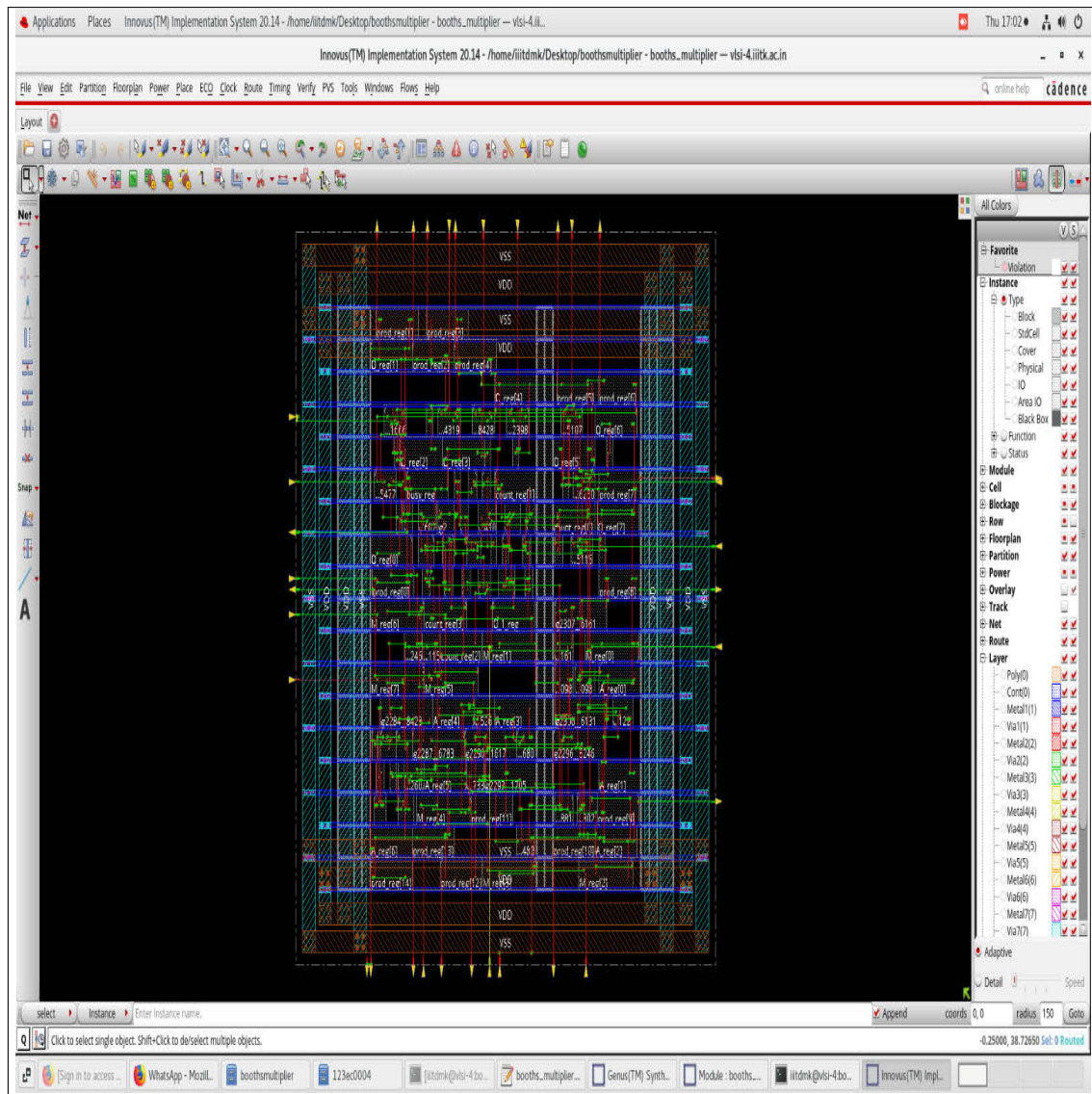


Figure 5: Cell Placed Layout

Routed Layout

Explanation: The routed layout completes the signal connections using multi-layer interconnects. This step ensures that all timing and electrical constraints are satisfied.

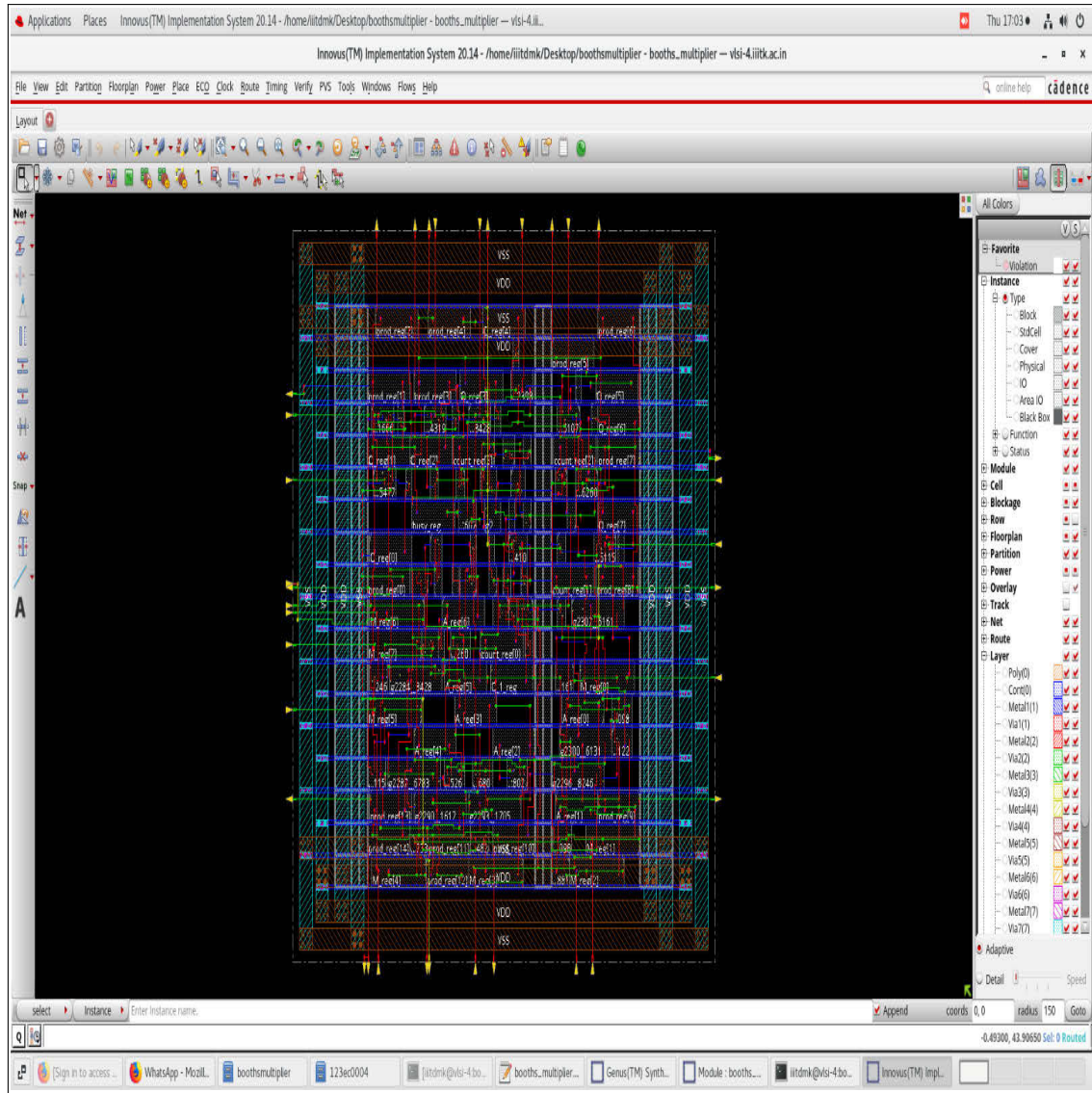


Figure 6: Routed Layout

Verilog Code

```
<module booth_multiplier (
    output reg [15:0] prod,
    output reg busy,
    input [7:0] mc, mp,
    input clk, start
);
    reg [7:0] A, Q, M;
    reg Q_1;
    reg [3:0] count;
    wire [7:0] sum, diff;

    always @(posedge clk) begin
        if (start) begin
            A <= 8'b0;
```



```

        M <= mc;
        Q <= mp;
        Q_1 <= 1'b0;
        count <= 4'b0;
        busy <= 1'b1;
    end else if (busy) begin
        case ({Q[0], Q_1})
            2'b01 : {A, Q, Q_1} <= {sum[7], sum, Q};
                // Add
            2'b10 : {A, Q, Q_1} <= {diff[7], diff, Q};
                // Subtract
            default : {A, Q, Q_1} <= {A[7], A, Q};
                // Shift only
        endcase
        count <= count + 1'b1;
        if (count == 4'd8)
            busy <= 1'b0;
    end
    prod <= {A, Q};
end

alu add1 (sum, A, M, 1'b0);
alu sub1 (diff, A, ~M, 1'b1);
endmodule

module alu (
    output [7:0] out,
    input [7:0] a, b,
    input cin
);
    assign out = a + b + cin;
endmodule
>

```

Testbench Code

```

<module tb_booth_multiplier();
    reg [7:0] mc, mp;
    reg clk, start;
    wire [15:0] prod;
    wire busy;

    booth_multiplier uut (.prod(prod), .busy(busy), .mc(mc),
        .mp(mp), .clk(clk), .start(start));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
end

```

```

        initial begin
            // Test Case 1
            mc = 8'd4; mp = 8'd3; start = 1; #10; start = 0;
            #80;
            // Test Case 2
            mc = 8'd15; mp = 8'd7; start = 1; #10; start = 0;
            #80;
            // Test Case 3 (Negative values)
            mc = -8'd4; mp = -8'd3; start = 1; #10; start = 0;
            #80;
            $stop;
        end
    endmodule
>

```

SDC Constraints

```

<# Set clock period to 10ns (100MHz)
create_clock -name clk -period 10 [get_ports clk]

# Input/output delay for primary inputs/outputs
set_input_delay 2 [get_ports mc]
set_input_delay 2 [get_ports mp]
set_output_delay 2 [get_ports prod]

# Set drive strength and load (optional, example values)
set_drive 4 [get_ports mc]
set_load 10 [get_ports prod]
>

```

TCL Script

```

<# Set up working directories
file mkdir netlist
file mkdir logs

# Read Verilog sources
read_verilog booth_multiplier.v
read_verilog tb_booth_multiplier.v

# Set top module for synthesis
set TOP booth_multiplier

# Run synthesis
synth_design -top $TOP

```

```
# Write synthesized netlist
write_verilog netlist/${TOP}_netlist.v

# Save reports
report_timing > logs/${TOP}_timing.rpt
report_area > logs/${TOP}_area.rpt
report_power > logs/${TOP}_power.rpt

# Simulate (optional, tool-dependent)
run_simulation tb_booth_multiplier.v

exit
>
```

3D View

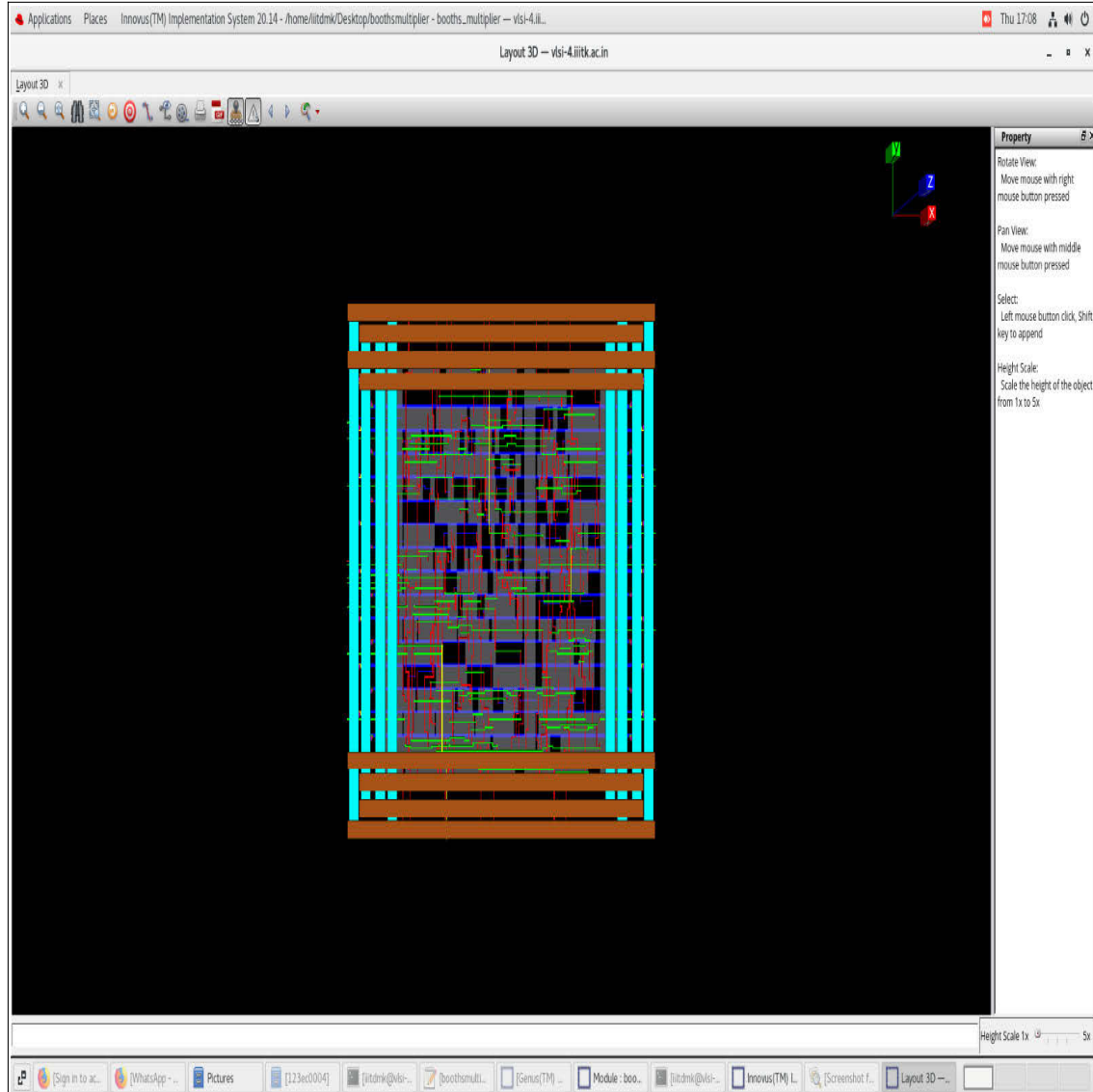


Figure 7: 3D Multi-Layer Routing View

Explanation: The 3D visualization shows multiple metal layers and vias. This verifies that the physical implementation follows DRC/LVS rules and is fabrication-ready.

Result

The 8-bit Booth's multiplier was successfully designed and implemented. Functional simulation, synthesis, place and route operations were completed. The circuit meets timing, area, and power constraints.

Conclusion

Booth's algorithm is an efficient technique for signed multiplication. Using a semi-custom VLSI design flow, the design was successfully taken from behavioral Verilog description to a physical layout suitable for fabrication. The experiment demonstrates the complete hardware realization process from RTL to GDSII generation.